Lab Schedule:

Tuesday/wednesday

Experiment 7

Experiment 8

Lab Schedule:

Saturday

Experiment 9

Experiment 10

Experiment 7

Decision Tree classifier using the ID3 algorithm

Implement a Decision Tree classifier using the ID3 algorithm to segment customers based on their purchasing behavior using the Online Retail dataset. Analyze the tree structure and discuss the feature importance.

Tasks:

- Load and preprocess the Online Retail dataset.
- Implement Decision Tree using the ID3 algorithm.
- Visualize the decision tree and analyze feature importance.
- Discuss how the tree structure helps in understanding customer behavior.

Theory

Decision Trees are supervised machine learning algorithms used for classification and prediction. The ID3 (Iterative Dichotomiser 3) algorithm constructs a tree using **Information Gain** as the splitting criterion.

Entropy

Entropy measures impurity in the dataset.

$$Entropy(S) = -\sum p_i \log_2(p_i)$$

where:

p_i= probability of class i

Information Gain

$$Gain(S, A) = Entropy(S) - \sum \frac{|\ S_v\ |}{|\ S\ |} Entropy(S_v)$$

The attribute with the highest information gain becomes the decision node.

Algorithm (ID3)

Step 1
Load the Online Retail dataset.
Step 2
Remove missing values.
Step 3
Create customer-level features:

- Total Quantity
- Total Spending
- Purchase Frequency

Step 4
Create customer segments:

- High Value
- Medium Value
- Low Value

Step 5
Calculate entropy of the target variable.
Step 6
Compute Information Gain for all features.
Step 7
Choose the feature with maximum Information Gain.
Step 8
Create decision nodes recursively until:

- All records belong to one class
- No attributes remain

Step 9
Visualize the decision tree.
Step 10
Evaluate feature importance.

Program

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Load Dataset
df = pd.read_excel("Online Retail.xlsx")

# Remove Missing Values
df = df.dropna()
```

```
# Create Total Amount
df["TotalAmount"] = df["Quantity"] * df["UnitPrice"]

# Customer Aggregation
customer_data = df.groupby("CustomerID").agg({
 "Quantity":"sum",
 "TotalAmount":"sum",
"InvoiceNo":"nunique"
}).reset_index()

customer_data.columns = [
"CustomerID",
"TotalQuantity",
"TotalSpending",
"PurchaseFrequency"
]

# Customer Segmentation
        customer_data["Segment"] = pd.qcut(
            customer_data["TotalSpending"],
            q=3,
            labels=["Low","Medium","High"]
        )

        # Features
        X = customer_data[
            ["TotalQuantity",
            "TotalSpending",
            "PurchaseFrequency"]
        ]

        # Target
        y = customer_data["Segment"]

        # Encode Target
        encoder = LabelEncoder()
        y = encoder.fit_transform(y)

        # Train Test Split
        X_train, X_test, y_train, y_test = train_test_split(
            X,
            y,
            test_size=0.2,
            random_state=42
        )
```

```
# ID3 Decision Tree
model = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42
)

model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Feature Importance
importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

print("\nFeature Importance")
print(importance)

# Tree Visualization
plt.figure(figsize=(16,8))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=encoder.classes_,
    filled=True
)

plt.show()
```

Experiment 8
Implement and compare Logistic Regression and Decision Trees

Aim

Implement and compare Logistic Regression and Decision Trees on the Adult Income dataset for predicting income levels. Evaluate both models based on performance metrics and interpretability.

Tasks:

● Load and preprocess the Adult Income dataset.
● Implement both Logistic Regression and Decision Trees.
● Compare the models based on metrics such as accuracy, precision, recall, and F1-score.
● Discuss the interpretability of both models and their suitability for the dataset.

Algorithm

Step 1
Load Adult Income dataset.
Step 2
Handle missing values.
Step 3
Encode categorical variables.
Step 4
Split dataset into training and testing sets.
Step 5
Train Logistic Regression model.
Step 6
Train Decision Tree model.
Step 7
Evaluate both models using:

- Accuracy
- Precision
- Recall
- F1-Score

Step 8
Compare model performance.
Step 9
Analyze interpretability.

Program

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

# Load Dataset
df = pd.read_csv("adult.csv")

# Handle Missing Values
df.replace(" ?", np.nan, inplace=True)
df.dropna(inplace=True)

# Encode Categorical Features
label_encoders = {}
for column in df.columns:
    if df[column].dtype == "object":
        le = LabelEncoder()
        df[column] = le.fit_transform(df[column])
        label_encoders[column] = le

# Features and Target
X = df.drop("income", axis=1)
y = df["income"]

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
# Logistic Regression
lr_model = LogisticRegression(max_iter=1000)

lr_model.fit(X_train, y_train)
lr_pred = lr_model.predict(X_test)

# Decision Tree
dt_model = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=5,
    random_state=42
)
dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)

# Evaluation Function
def evaluate(y_true, y_pred, model_name):
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)

    print("\n", model_name)
    print("-"*30)
    print("Accuracy :", accuracy)
    print("Precision:", precision)
    print("Recall  :", recall)
    print("F1 Score :", f1)

# Logistic Regression Results
evaluate(y_test, lr_pred, "Logistic Regression")

# Decision Tree Results
evaluate(y_test, dt_pred, "Decision Tree")
print("\nClassification Report - Logistic Regression")
print(classification_report(y_test, lr_pred))
print("\nClassification Report - Decision Tree")
print(classification_report(y_test, dt_pred))

# Decision Tree Visualization
plt.figure(figsize=(18,10))
plot_tree(
    dt_model,
    feature_names=X.columns,
    filled=True,
    fontsize=8
```

```
)
plt.show()
```

Experiment 9
Linear Support Vector Machine (SVM) to classify the Iris dataset

Aim:

Implement a Linear Support Vector Machine (SVM) to classify the Iris dataset. Visualize the decision boundary and discuss how the margin is determined.

Tasks:

- Load and preprocess the Iris dataset.
- Implement a Linear SVM for binary classification
- Visualize the decision boundary and margin.
- Discuss the concept of the margin and how it influences classification.

Theory

Support Vector Machine (SVM)

Support Vector Machine is a supervised machine learning algorithm used for classification and regression.

The main objective of SVM is to find the optimal hyperplane that maximizes the separation between classes.

Hyperplane Equation

$$w^T x + b = 0$$

where:

- w= weight vector
- x= feature vector
- b= bias

Margin

The margin is the distance between:

- The decision boundary

- The nearest data points from each class
  These nearest points are called Support Vectors.
  Margin Formula

$$Margin = \frac{2}{|| w ||}$$

  A larger margin generally leads to better generalization and reduced overfitting.

Algorithm

Step 1
Load the Iris dataset.
Step 2
Select two features for visualization:

- Petal Length
- Petal Width

Step 3
Convert the target variable:
Setosa → 1
Non-Setosa → 0

Step 4
Split dataset into training and testing sets.
Step 5
Train Linear SVM classifier.
Step 6
Predict test samples.
Step 7
Evaluate model performance.
Step 8
Plot:

- Data points
- Decision boundary
- Margin boundaries
- Support vectors

Step 9
Analyze margin and classification behavior.

Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report

# Load Iris Dataset
iris = datasets.load_iris()

# Use only Petal Length and Petal Width
X = iris.data[:, [2, 3]]

# Binary Classification
# Setosa = 1
# Non-Setosa = 0
y = (iris.target == 0).astype(int)

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

# Linear SVM
svm_model = SVC(
    kernel='linear',
    C=1
)

svm_model.fit(X_train, y_train)

# Prediction
y_pred = svm_model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print("\nClassification Report")
print(classification_report(y_test, y_pred))
```

```
# Plot Decision Boundary
plt.figure(figsize=(8,6))
plt.scatter(
    X[:,0],
    X[:,1],
    c=y,
    cmap='coolwarm'
)

ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()
xx = np.linspace(xlim[0], xlim[1], 30)
yy = np.linspace(ylim[0], ylim[1], 30)
YY, XX = np.meshgrid(yy, xx)
xy = np.vstack([XX.ravel(), YY.ravel()]).T
Z = svm_model.decision_function(xy)
Z = Z.reshape(XX.shape)

# Decision Boundary
ax.contour(
    XX,
    YY,
    Z,
    colors='k',
    levels=[-1, 0, 1],
    alpha=0.8,
    linestyles=['--','-','--']
)

# Support Vectors
ax.scatter(
    svm_model.support_vectors_[:,0],
    svm_model.support_vectors_[:,1],
    s=100,
    linewidth=1,
    facecolors='none',
    edgecolors='black'
)
plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.title("Linear SVM Decision Boundary and Margin")
plt.show()
```

Experiment 10

Implement and compare the performance of SVM classifiers with linear, polynomial, and RBF kernels

Aim:

Implement and compare the performance of SVM classifiers with linear, polynomial, and F kernels on the Fashion MNIST dataset. Analyze the advantages and disadvantages of each kernel type.

Tasks:

- Load and preprocess the Fashion MNIST dataset.
- Implement SVM with linear, polynomial, and RBF kernels.
- Compare the classification performance for each kernel.
- Discuss the strengths and weaknesses of each kernel type.

Theory

Support Vector Machine (SVM)

Support Vector Machine is a supervised machine learning algorithm that finds an optimal hyperplane to separate classes while maximizing the margin between them.

Decision Function

$$f(x) = w^T x + b$$

For nonlinear datasets, SVM uses kernel functions to transform data into higher-dimensional spaces.

Kernel Functions

1. Linear Kernel

Used when data is approximately linearly separable.

$$K(x_i, x_j) = x_i^T x_j$$

2. Polynomial Kernel

Maps data into polynomial feature space.

$$K(x_i, x_j) = (x_i^T x_j + c)^d$$

where:

- d= polynomial degree
- c= constant

3. RBF Kernel

The Radial Basis Function kernel is the most widely used nonlinear kernel.

$$K(x_i, x_j) = e^{-\gamma ||x_i - x_j||^2}$$

where:

- γcontrols influence of training samples

Algorithm

Step 1: Load Fashion MNIST dataset.
Step 2: Normalize pixel values.
Step 3: Flatten images into vectors.
Step 4: Split data into training and testing sets.
Step 5: Train Linear SVM classifier.
Step 6: Train Polynomial Kernel SVM classifier.
Step 7: Train RBF Kernel SVM classifier.
Step 8: Predict test data.
Step 9: Calculate:

- Accuracy
- Precision
- Recall
- F1-score

Step 10: Compare performance of all kernels.

Program

```
import numpy as np
import pandas as pd
from tensorflow.keras.datasets import fashion_mnist
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report

# Load Fashion MNIST
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

# Normalize Images
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten Images
X_train = X_train.reshape(len(X_train), -1)
X_test = X_test.reshape(len(X_test), -1)

# To reduce training time
X_train = X_train[:10000]
y_train = y_train[:10000]
X_test = X_test[:2000]
y_test = y_test[:2000]
```

```
# Linear Kernel
linear_svm = SVC(
    kernel='linear'
)

linear_svm.fit(X_train, y_train)

linear_pred = linear_svm.predict(X_test)

# Polynomial Kernel
poly_svm = SVC(
    kernel='poly',
    degree=3
)

poly_svm.fit(X_train, y_train)
poly_pred = poly_svm.predict(X_test)

# RBF Kernel
rbf_svm = SVC(
    kernel='rbf'
)

rbf_svm.fit(X_train, y_train)
rbf_pred = rbf_svm.predict(X_test)

# Evaluation Function
def evaluate_model(y_true, y_pred, model_name):
    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(
        y_true,
        y_pred,
        average='weighted'
    )
    recall = recall_score(
        y_true,
        y_pred,
        average='weighted'
    )
    f1 = f1_score(
        y_true,
        y_pred,
        average='weighted'
    )
    print("\n", model_name)
    print("-"*40)
    print("Accuracy :", accuracy)
    print("Precision:", precision)
```

```
    print("Recall  :", recall)
    print("F1 Score :", f1)


# Results
evaluate_model(y_test, linear_pred, "Linear SVM")
evaluate_model(y_test, poly_pred, "Polynomial SVM")
evaluate_model(y_test, rbf_pred, "RBF SVM")
```